Signed p-adic Regression Encodings of Finite-Domain All-Different Systems with a Sudoku Case Study

Greg Baker
School of Computing, Australian National University
Canberra, Australia
greg.baker@anu.edu.au

Abstract

This note studies a signed synthetic p-adic regression encoding of finite-domain all-different systems and uses Sudoku as a worked example. Given a graph whose vertices carry finite allowed sets and whose edges reward unequal assignments, we build positive synthetic observations that create unary domain wells and negative synthetic observations that reward pairwise inequality. For primes $p > q$, every nonzero difference between values in $\{1, \ldots, q\}$ has p-adic norm 1, so on domain-respecting states the loss is an additive constant plus the edge-conflict count. It follows that the global minimisers are exactly the domain-respecting assignments that maximise the number of unequal edges. When the associated list-colouring instance is satisfiable, the minimisers are precisely its proper list-colourings.

Standard Sudoku is an immediate special case: there is one variable per cell, singleton domains on clue cells, and edges between cells that share a row, column, or box. The resulting objective lives on the native 81-cell digit state space rather than on a one-hot lift to 729 Boolean indicators. We also record two small local-search heuristics, including a discrete Zubarev-style random walk, to show that the encoding can be operated on directly in the Sudoku grid. These computational experiments are purely illustrative and are not intended as competitive Sudoku methods. The main contribution is the general encoding theorem and its finite-domain all-different corollary.

Keywords: p-adic regression; all-different constraints; list colouring; synthetic datasets; Sudoku

1 Introduction and prior work

There are many sensible ways to solve Sudoku puzzles. Exact-cover formulations solved by Knuth's Algorithm X with dancing links [3], constraint-programming formulations built around *alldifferent* constraints [4], and SAT encodings [5] are all natural approaches. This paper is not one of them.

Instead, the paper asks whether finite-domain all-different systems can be encoded as a single signed synthetic p-adic regression objective on digit-valued variables rather than one-hot indicators, with Sudoku as a worked example. The point is not computational efficiency but representational parsimony: use one variable per semantic object, keep the domain of each variable aligned with the problem domain, and let a single p-adic objective enforce both domain membership and pairwise distinctness. On digit-valued states the loss becomes a constant plus a visible conflict count, so local updates can be followed in the native Sudoku grid rather than through an auxiliary encoding.

This is the sense in which the construction is intended to be hand-traceable. The claim is not that a human would solve Sudoku by imitating the optimiser; it is that once the state is represented by digits directly, the effect of a proposed local move can be checked by hand from the affected units alone.

Two earlier results motivate the construction:

- A structural result on p-adic linear regression [1]: an optimal hyperplane is supported by $n+1$ data points.
- A complexity result on p-adic linear regression [2]: the problem is NP-hard, via an embedding of MAX-CUT using positive regularisation points that snap coefficients to $\{0,1\}$.

The main contribution here is the general signed construction in Section 3: negative regularisation is used to enforce pairwise distinctness across arbitrary finite domains, and Sudoku serves as a concrete case study that makes the native-state and locally auditable character of the encoding explicit.

2 Preliminaries

2.1 p-adic integers and norms

Fix a prime p. The p-adic valuation $v_p(n)$ of an integer $n \neq 0$ is the exponent of p in its factorisation, and the p-adic absolute value is

$$|n|_p = p^{-v_p(n)}, \qquad |0|_p = 0.$$

We will work in the p-adic integers $\mathbb{Z}_p$, where all norms satisfy $|x|_p \leq 1$. The key choice for Sudoku is to take $p > 9$ (e.g. $p = 11$), so that the p-adic absolute value of any Sudoku digit or digit difference is always 0 or 1.

3 A general all-different regression template

3.1 List colouring

Many finite-domain constraint systems built from all-different conditions can be viewed as list-colouring problems on a graph. Each variable x_i is assigned a value from a finite allowed set D_i, and an edge $\{i,j\}$ expresses the preference or requirement that $x_i \neq x_j$.

Figure 1 gives a concrete toy instance. Its synthetic dataset has six positive observations, one for each allowed value in the three domains, and three negative observations, one for each edge. The proper list-colouring $x_1 = 1$, $x_2 = 2$, $x_3 = 3$ therefore activates every edge reward while incurring only the unavoidable unary baseline.

3.2 Regression formulation

To make the regression viewpoint explicit, it is useful to describe the *synthetic dataset* rather than only the resulting loss. In the present construction there is no ambient collection of ordinary

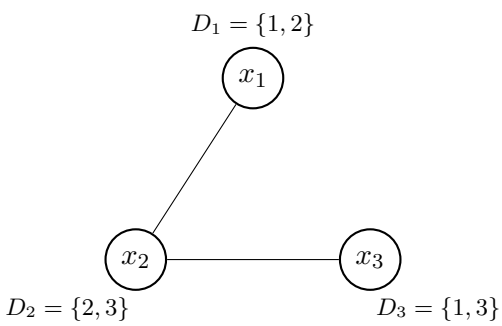

Figure 1: A tiny list-colouring instance. The assignment $x_1 = 1$, $x_2 = 2$, $x_3 = 3$ respects the domains and makes every edge unequal.

Type	Synthetic observations	Role
positive	$(+1, e_1, a, \lambda)$ for $a \in \{1, 2\}$	pin x_1 to D_1
positive	$(+1, e_2, a, \lambda)$ for $a \in \{2, 3\}$	pin x_2 to D_2
positive	$(+1, e_3, a, \lambda)$ for $a \in \{1, 3\}$	pin x_3 to D_3
negative	$(-1, e_1 - e_2, 0, 1)$	reward $x_1 \neq x_2$
negative	$(-1, e_1 - e_3, 0, 1)$	reward $x_1 \neq x_3$
negative	$(-1, e_2 - e_3, 0, 1)$	reward $x_2 \neq x_3$

Figure 2: Synthetic dataset for the toy list-colouring instance in Figure 1.

feature-response pairs. Every observation is inserted deliberately, either to pin a variable to its allowed values or to reward separation along an edge.

For the toy instance in Figure 1, this equivalence can be written out explicitly. Figure 2 shows the corresponding synthetic dataset: six positive observations encode the three domains, and three negative observations encode the three edges.

Over $\mathbb{Z}_p$ with $p > q$, the p-adic norm is perfectly suited to this setting: once values are restricted to $\{1, \ldots, q\}$, every nonzero difference has norm 1.

Definition 1 (Signed synthetic p-adic regression dataset)**.** *For parameters $x \in \mathbb{Z}_p^n$, a signed weighted affine observation is a quadruple (σ, u, b, γ) with $\sigma \in \{+1, -1\}$, $u \in \mathbb{Z}^n$, $b \in \mathbb{Z}$, and $\gamma > 0$. It contributes the term*

$$\sigma\gamma\, |u^\top x - b|_p$$

to the loss; formally, this is the p-adic analogue of a weighted Euclidean residual term.

Definition 2 (Positive/negative observations)**.** *A signed weighted affine observation (σ, u, b, γ) is* positive *if $\sigma = +1$ and* negative *if $\sigma = -1$. We also use the phrases* positive regularisation term *and* negative regularisation term*.*

Definition 3 (Synthetic dataset for list-colouring)**.** *Let $V = \{1, \ldots, n\}$, let $G = (V, E)$ be a finite graph, let $D_i \subseteq \{1, \ldots, q\}$ be nonempty allowed sets, and fix a prime $p > q$. Define*

$$w_{ii} := 0, \qquad w_{ij} := \begin{cases} 1, & \{i, j\} \in E, \\ 0, & \{i, j\} \notin E, \end{cases} \qquad (i \neq j),$$

let $\Delta := \max_{i \in V} \sum_{j=1}^n w_{ij}$, and set $\lambda := 1 + \Delta$. Writing $e_1, \ldots, e_n$ for the standard basis vectors of $\mathbb{Z}^n$, define

$$\mathcal{S}^+ := \{(+1, e_i, a, \lambda) : i \in V,\ a \in D_i\},$$

$$\mathcal{S}^- := \{(-1, e_i - e_j, 0, 1) : 1 \leq i < j \leq n,\ w_{ij} = 1\}.$$

Set $\mathcal{S} := \mathcal{S}^+ \sqcup \mathcal{S}^-$.

Definition 4 (Loss associated with the list-colouring dataset)**.** *For $\mathcal{S}$ as in Definition 3, define*

$$L_{\mathcal{S}}(x) = \lambda \sum_{i=1}^n \sum_{a \in D_i} |x_i - a|_p - \sum_{1 \leq i < j \leq n} w_{ij} |x_i - x_j|_p.$$

We now analyse that construction. This dataset has exactly $\sum_{i=1}^n |D_i|$ positive observations and $|E|$ negative observations, so it is computable in time $O(\sum_i |D_i| + |E|)$. The positive observations pin coordinates to allowed values, the negative observations reward inequality across edges, and there are no ordinary data points unrelated to the constraint system.

Each positive observation $(+1, e_i, a, \lambda)$ contributes $\lambda |x_i - a|_p$ to the loss, and each negative observation $(-1, e_i - e_j, 0, 1)$ contributes $-|x_i - x_j|_p$ to the loss. Summing them yields the displayed loss. The size bound and runtime are immediate from the explicit definitions.

3.3 Correctness

Since $p > q$, the residues of $1, \ldots, q$ are distinct modulo p. In particular, for any fixed $i \in V$ and any $t \in \mathbb{Z}_p$, there is at most one $a \in D_i$ with $t \equiv a \pmod{p}$.

First we record the simple observation that, on $\{1, \ldots, q\}$, the p-adic norm detects inequality exactly.

Lemma 1 (Edge terms become inequality indicators on the domain)**.** *If $x_i, x_j \in \{1, \ldots, q\}$ and $p > q$, then*

$$|x_i - x_j|_p = \begin{cases} 0, & x_i = x_j, \\ 1, & x_i \neq x_j. \end{cases}$$

Proof. If $x_i = x_j$ then the difference is zero. If $x_i \neq x_j$, then $1 \leq |x_i - x_j| \leq q - 1 < p$, so $p \nmid (x_i - x_j)$ and therefore $|x_i - x_j|_p = 1$. □

We next prove a digit-snapping lemma: if x minimises $L_{\mathcal{S}}$, then each coordinate x_i lies in D_i. The explicit unary-well formula makes that argument transparent.

Lemma 2 (Unary wells)**.** *For $i \in V$, define*

$$U_i(t) := \sum_{a \in D_i} |t - a|_p \qquad (t \in \mathbb{Z}_p).$$

Then $U_i(t) \geq |D_i| - 1$, with equality if and only if $t \in D_i$.

Proof. If $t \in D_i$, then one summand in $U_i(t)$ is 0, and the remaining $|D_i| - 1$ summands are 1, so $U_i(t) = |D_i| - 1$.

If $t \notin D_i$ but $t \equiv b \pmod{p}$ for some $b \in D_i$, then $0 < |t - b|_p < 1$, while for every $a \in D_i \setminus \{b\}$ we have $t - a \equiv b - a \not\equiv 0 \pmod{p}$, so $|t - a|_p = 1$. Hence

$$U_i(t) = (|D_i| - 1) + |t - b|_p > |D_i| - 1.$$

If t is congruent modulo p to no element of D_i, then $|t - a|_p = 1$ for all $a \in D_i$, so $U_i(t) = |D_i| > |D_i| - 1$. □

Lemma 3 (Digit-snapping lemma)**.** *Let $x \in \mathbb{Z}_p^n$. Suppose there exists $i \in V$ with $x_i \notin D_i$. Choose $b \in D_i$ as follows: if x_i is congruent modulo p to an element of D_i, take that unique element; otherwise choose any $b \in D_i$. Let x' be obtained from x by replacing x_i with b. Then*

$$L_{\mathcal{S}}(x') < L_{\mathcal{S}}(x).$$

Proof. The positive part of the loss changes by

$$\lambda \left(\sum_{a \in D_i} |b - a|_p - \sum_{a \in D_i} |x_i - a|_p \right).$$

There are two cases to consider.

If x_i is congruent modulo p to no element of D_i, then $|x_i - a|_p = 1$ for every $a \in D_i$, so $\sum_{a \in D_i} |x_i - a|_p = |D_i|$. On the other hand, one summand in $\sum_{a \in D_i} |b - a|_p$ is 0 and the remaining $|D_i| - 1$ summands are 1, so $\sum_{a \in D_i} |b - a|_p = |D_i| - 1$. Hence the positive change is $-\lambda$. Since $|x_i - b|_p = 1$ in this case, this is at most $-\lambda |x_i - b|_p$.

If $x_i \equiv b \pmod{p}$ for some $b \in D_i$, then $\sum_{a \in D_i} |b - a|_p = |D_i| - 1$, whereas

$$\sum_{a \in D_i} |x_i - a|_p = (|D_i| - 1) + |x_i - b|_p.$$

Hence the positive change is $-\lambda|x_i - b|_p$.

For each $j \in V$, the corresponding negative component changes by

$$w_{ij}\big(|x_i - x_j|_p - |b - x_j|_p\big).$$

By the ultrametric inequality,

$$|x_i - x_j|_p \le \max\big(|x_i - b|_p,\ |b - x_j|_p\big),$$

so

$$|x_i - x_j|_p - |b - x_j|_p \le |x_i - b|_p.$$

Summing over all $j \in V$,

$$L_{\mathcal{S}}(x') - L_{\mathcal{S}}(x) \le \left(-\lambda + \sum_{j=1}^{n} w_{ij}\right) |x_i - b|_p \le -|x_i - b|_p < 0.$$

□

Next we analyse the loss on domain-respecting assignments. In that regime it becomes a constant plus a conflict count, so if the list-colouring instance is satisfiable the rightmost sum can be zero.

Theorem 1 (Correctness of the synthetic reduction)**.** *For the dataset $\mathcal{S} = \mathcal{S}^+ \sqcup \mathcal{S}^-$ from Definition 3 and the loss from Definition 4, every global minimiser of $L_{\mathcal{S}}$ lies in $\prod_{i=1}^n D_i$. On $\prod_{i=1}^n D_i$,*

$$L_{\mathcal{S}}(x) = \lambda \sum_{i=1}^{n} (|D_i| - 1) - |E| + \sum_{1 \le i < j \le n} w_{ij} \mathbf{1}_{x_i = x_j}.$$

Consequently, the global minimisers are exactly the domain-respecting assignments that minimise the conflict sum $\sum_{1 \le i < j \le n} w_{ij} \mathbf{1}_{x_i = x_j}$, or equivalently maximise the number of unequal edges. In particular, if the list-colouring instance $(G, (D_i)_{i=1}^n)$ is satisfiable, then this minimum conflict sum is 0, and the global minimisers are exactly its proper list-colourings.

Proof. By Lemma 3, any point with some coordinate outside its domain can be improved by snapping that coordinate into the domain. Therefore every global minimiser lies in $\prod_i D_i$. If $x \in \prod_i D_i$, then Lemma 2 gives $U_i(x_i) = |D_i| - 1$ for every i, and Lemma 1 gives

$$|x_i - x_j|_p = \mathbf{1}_{x_i \ne x_j} \qquad (1 \le i < j \le n).$$

Since $\mathbf{1}_{x_i \ne x_j} = 1 - \mathbf{1}_{x_i = x_j}$ on every edge, substituting these identities into $L_{\mathcal{S}}$ yields the displayed formula. The final claims follow immediately. □

Corollary 2. *Any finite-domain all-different constraint system can be encoded, in polynomial time, as the global minimiser set of a signed synthetic p-adic regression dataset.*

Proof. Given an all-different constraint system, form its primal graph: the vertices are the variables, and two variables are adjacent if they appear together in some all-different constraint. The satisfying assignments are exactly the proper list-colourings of this graph, with the variable domains as lists. Apply Definitions 3 and 4, and Theorem 1. □

4 Sudoku as a special case

4.1 Sudoku

A (standard) Sudoku puzzle consists of a partially filled 9×9 grid. The task is to assign a digit in $\{1, \ldots, 9\}$ to each empty cell such that:

1. each row contains all digits $1, \ldots, 9$ exactly once;
2. each column contains all digits $1, \ldots, 9$ exactly once; and
3. each 3×3 box contains all digits $1, \ldots, 9$ exactly once.

Equivalently, in each unit (row, column, box) all entries are *pairwise distinct* and each entry lies in $\{1, \ldots, 9\}$.

Standard Sudoku fits Corollary 2 directly. The vertices are the 81 cells, two vertices are adjacent when the corresponding cells share a row, column, or 3×3 box, unclued cells have domain $\{1, \ldots, 9\}$, and clue cells have singleton domains. A proper list-colouring of this graph is exactly a valid Sudoku completion.

The general lemmas above specialise very cleanly to Sudoku. We therefore state the Sudoku facts below mainly as direct corollaries of the unary-well and edge-indicator lemmas, and only then write down the concrete synthetic datasets used in the paper.

4.2 Positive regularisation: snapping coefficients to digits

Let $D = \{1, 2, \ldots, 9\} \subset \mathbb{Z}_p$, and write the Sudoku variables as a vector

$$x \in \mathbb{Z}_p^{81},$$

with one coefficient per cell.

Definition 5 (Sudoku digit-snapping penalty)**.** *For $p > 9$, define the digit-snapping penalty*

$$U_D(t) := \sum_{k=1}^{9} |t - k|_p.$$

By Lemma 2 applied to the common domain D, one has $U_D(t) \geq 8$ for every $t \in \mathbb{Z}_p$, with equality if and only if $t \in D$.

This is the 9-valued analogue of the earlier "snap to $\{0, 1\}$" construction from [2]: the synthetic observations $t = k$ for all $k \in D$ create a multiwell objective with minima exactly at D.

4.3 Negative regularisation: rewarding inequality via p-adic norms

The remaining Sudoku constraints are that entries in each row/column/box are pairwise unequal. Pairwise inequality can be encoded using differences $x_i - x_j$.

Corollary 3 (Sudoku differences as inequality indicators)**.** *Let $p > 9$ and $a, b \in D$. Then*

$$|a - b|_p = \begin{cases} 0 & a = b, \\ 1 & a \neq b. \end{cases}$$

Proof. This is Lemma 1 with $q = 9$. □

4.4 Minimal and expanded regression objectives

Let $\mathcal{P}$ be the set of cell-pairs (i, j) that share a unit (same row, same column, or same box). Let $\mathcal{C}$ be the set of given clues; for $i \in \mathcal{C}$ let $g_i \in D$ be the fixed digit.

Specialising Definitions 3 and 4, together with Theorem 1, to the Sudoku graph gives the minimal synthetic dataset

$$L_{\min}(x) = \alpha \sum_{i \notin \mathcal{C}} \sum_{k=1}^{9} |x_i - k|_p \; + \; \alpha \sum_{i \in \mathcal{C}} |x_i - g_i|_p \; - \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \tag{1}$$

A separate coefficient on the pairwise term is unnecessary here: any positive edge weight can be scaled out, so we normalise that coefficient to 1. Thus α itself plays the role of the general pinning weight λ, and any choice $\alpha > 20$ satisfies the domination condition from Theorem 1. Every term in (1) is a regularisation term with a specific purpose: the positive terms pin cells to allowed digits or clues, and the negative terms reward unequal peer values. There are no ordinary feature-response observations beyond these synthetic regularisers. The minimal dataset therefore contains $9(81 - |\mathcal{C}|) + |\mathcal{C}| = 729 - 8|\mathcal{C}|$ positive observations and 810 negative observations. This is the conceptual formulation used in the correctness statements and in the parsimony discussion below: a single scalar objective on the native digit state space.

For the experiments, however, we use the slightly more redundant expanded objective

$$L(x) = \alpha \sum_{i=1}^{81} \sum_{k=1}^{9} |x_i - k|_p \; + \; \alpha_{\text{clue}} \sum_{i \in \mathcal{C}} |x_i - g_i|_p \; - \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p. \tag{2}$$

Equation (2) keeps the nine digit-well residuals visible even on clue cells and then adds the clue pins separately. It therefore contains $729 + |\mathcal{C}|$ positive observations and the same 810 negative observations. The first two sums are *positive* regularisation, and the final sum is *negative* regularisation. There is still no "ordinary" data in the usual regression sense: every synthetic observation exists only to enforce a Sudoku constraint. Equivalently, counting peer pairs per unit gives $27\binom{9}{2} = 972$ negative terms, but 162 of these pairs lie simultaneously in a row/column *and* a box, leaving 810 distinct pairs overall.

This expanded form is useful for implementation, but its unrestricted correctness requires an explicit clue-weight assumption. On a clue cell, the first sum in (2) has the same value 8α for every digit $x_i \in D$, so clue fidelity is enforced only by the separate term $\alpha_{\text{clue}}|x_i - g_i|_p$. Accordingly, the same domination argument as before applies to (2) when $\alpha_{\text{clue}} > 20$; the simplest choice is $\alpha_{\text{clue}} = \alpha = 21$. Under that extra condition, (2) may be read as a stronger implementation-oriented refinement of (1), not as the more conceptually fundamental model.

Once x is digit-valued, the pairwise part is simply

$$- \sum_{(i,j) \in \mathcal{P}} |x_i - x_j|_p = -|\mathcal{P}| + \sum_{(i,j) \in \mathcal{P}} \mathbf{1}_{x_i = x_j},$$

so the loss differs from the peer-conflict count only by an additive constant.

The 810 pairwise terms are essentially irreducible if one insists on *pairwise* penalties: omitting a single peer-pair (i, j) removes the only residual that directly distinguishes $x_i = x_j$ from $x_i \neq x_j$ once x is digit-valued, and no linear "hash" of the remaining differences can substitute because nonvanishing is not preserved under linear combinations (e.g. $a - b \neq 0$ and $b - c \neq 0$ does not prevent $a - c = 0$).

Corollary 4 (Sudoku completion as a special case)**.** *Let G_{Sud} be the graph on the 81 cells of a Sudoku puzzle, with an edge between two cells exactly when they share a row, column, or 3×3 box. For a puzzle with clue set $\mathcal{C}$ and clue digits g_i, define*

$$D_i := \begin{cases} \{g_i\}, & i \in \mathcal{C}, \\ \{1, \ldots, 9\}, & i \notin \mathcal{C}. \end{cases}$$

Assume $\alpha > 20$. Then every global minimiser of the minimal objective (1) *is a clue-respecting digit assignment that minimises the peer-conflict count. In particular, if the Sudoku instance is satisfiable, the global minimisers are exactly the valid completions of the puzzle.*

Proof. Each row, column, and box of Sudoku forms a clique of size 9 in G_{Sud}. Thus a proper list-colouring assigns pairwise distinct digits in each unit. Since the available values are exactly $\{1, \ldots, 9\}$, pairwise distinctness in a unit is equivalent to containing every digit exactly once. The singleton domains enforce the clues. Apply Theorem 1. □

For Sudoku, this bookkeeping is completely explicit. In the deduplicated peer graph, every cell has degree 20, so Theorem 1 requires $\alpha > 20$ for the minimal objective (1); a convenient sufficient choice is $\alpha = 21$. For the expanded objective (2), the corresponding unrestricted claim requires $\alpha_{\mathrm{clue}} > 20$ as well, because the all-digit wells are constant on clue cells. If instead one rewrote the loss as a sum over row, column, and box scopes and counted overlaps separately, each cell would contribute $8 + 8 + 8 = 24$ incident negative terms, and the corresponding sufficient choice would be > 24. This is the precise place where the negative regularisation has to be dominated by the positive unary weight.

Why p-adics help with negative weights. If one tries to do this with an ordinary squared-loss term $-(x_i - x_j)^2$ in $\mathbb{R}$, the optimiser is perversely incentivised to make some differences *large*, not merely nonzero, and the objective can become unstable unless heavily bounded. With $p > 9$ and digit-valued variables, $|x_i - x_j|_p \in \{0, 1\}$, removing magnitude bias.

5 Illustrative local-search heuristics

The objective (2) induces an NP-hard global optimisation problem, consistent with the earlier NP-hardness result for p-adic regression. The computational material in this paper is included only to show that the encoding can be operated on directly; it is not intended as a competitive Sudoku method or a benchmark study.

In the experiments we work directly in the digit-valued, clue-consistent row-permutation state space. There the two positive sums in (2) are constant, so minimising L is equivalent to minimising the remaining column/box conflict count. Concretely, we initialise each row as a random permutation of $1, \ldots, 9$ consistent with its clues, and then apply row-wise swaps that reduce the remaining loss. This is best described as a block-local search on the integer state space.

We will call this move set *hand-traceable*: the state is a filled Sudoku grid, and the effect of a candidate swap can be checked by inspecting only the units that the swap touches, rather than by decoding an auxiliary indicator representation.

Lemma 4 (Locality of row swaps)**.** *Let x be a digit-valued, clue-consistent Sudoku state in which each row is a permutation of $1, \ldots, 9$. Let ξ swap two non-clue entries in a single row r, at columns $c_1 \neq c_2$. Then the change*

$$\Delta L(\xi) := L(x \oplus \xi) - L(x)$$

is determined entirely by the conflict-pair contributions from columns c_1 and c_2, together with the one or two boxes containing (r, c_1) and (r, c_2). No other term of the digit-restricted objective changes.

Proof. On digit-valued clue-consistent states, the positive part of (2) is constant, so only the column/box conflict count matters. A within-row swap preserves the row permutation, so row conflicts remain zero. Every cell outside row r is unchanged, and within row r only the two swapped positions change value. Hence only the two corresponding columns can change

Algorithm 1 Row-swap local search

Require: Puzzle with clues $\mathcal{C}$, max steps T, restarts R

```
for r = 1 to R do
    Initialise each row as a permutation of 1, ..., 9 consistent with its clues
    for t = 1 to T do
        if no column/box conflicts then
            return solution
        end if
        Choose a row involved in a conflict
        Find the best swap of two non-clue cells in that row (largest loss decrease)
        Apply the swap (or a random swap if no improving swap exists)
    end for
end for
return best assignment found
```

their conflict counts. Likewise, only the boxes containing those two cells can change. All other columns and boxes see the same multiset of digits before and after the swap, so their contributions are unchanged. □

This "stepwise regression" view matches the informal idea: at each step, choose the coefficient change that gives the largest incremental loss improvement. Lemma 4 is the precise bridge from the p-adic formulation to hand-traceability: in the row-permutation regime, a move is scored by a small local audit of the affected columns and boxes.

On the standard example puzzle used in the code, this row-swap heuristic reaches a solution after 228 swaps for seed 0 on restart 0. The only fact used later is the locality statement above: each move is scored by inspecting only the affected columns and boxes.

5.1 A discrete Zubarev walk

The row-swap heuristic above is tailored to the strong-snapping Sudoku setting. It is also natural to test a more generic training dynamic from the p-adic regression literature. In [7], Zubarev proposes a random-walk-based optimisation scheme for loss functions built from p-adic norms, precisely because such losses are locally constant almost everywhere and hence poorly served by gradient methods.

Concretely, the proposed training dynamics is a discrete-time random process

$$w_{t+1} = w_t + \xi_t(w_t, \beta_t), \tag{3}$$

where $\beta_t > 0$ is a "continuity" (inverse temperature) parameter and ξ_t is sampled from a distribution biased toward loss-decreasing moves. In Zubarev's formulation the conditional law has density (with respect to the p-adic Haar measure)

$$P(\xi \mid w, \beta) \;\propto\; \exp\bigl(-\beta\,(L(w+\xi) - L(w))\bigr). \tag{4}$$

Under mild conditions (in particular, that the set of improving moves has nonzero measure), Zubarev shows there is a regime of β values for which the expected loss decreases, i.e. $L(w_t)$ is a strict supermartingale [7].

In the digit-restricted implementation used here, the state is already digit-valued and clue-consistent. We therefore treat the Zubarev walk as a *finite-move* stochastic optimiser:

- The parameter vector w is the current Sudoku filling $x \in D^{81}$.
- The loss $L(x)$ is the number of conflicting pairs in columns and boxes (equivalently, (2) up to constants in the digit-restricted regime).

Algorithm 2 Zubarev walk over row swaps (heuristic)

Require: Puzzle with clues $\mathcal{C}$, max steps T, restarts R, schedule $\{\beta_t\}_{t=1}^{T}$

```
for r = 1 to R do
    Initialise each row as a permutation of 1, ..., 9 consistent with its clues
    for t = 1 to T do
        if no column/box conflicts then
            return solution
        end if
        Choose a row involved in a conflict
        Enumerate row swaps Ξ_r(x) and compute ΔL(ξ) for each ξ ∈ Ξ_r(x)
        Sample ξ ∈ Ξ_r(x) with probability ∝ exp(−β_t ΔL(ξ))
        Apply swap ξ
    end for
end for
return best assignment found
```

- A move ξ is a swap of two non-clue cells within a single row (preserving each row as a permutation of $1, \ldots, 9$).

For a chosen row r and current state x, let $\Xi_r(x)$ be the set of all such swaps, and let $\Delta L(\xi) := L(x \oplus \xi) - L(x)$ be the change in loss after applying swap ξ (where $x \oplus \xi$ denotes "apply the swap"). Replacing the Haar measure in (4) with the counting measure on the finite set $\Xi_r(x)$ gives the discrete sampling rule

$$\Pr(\xi \mid x, \beta_t) = \frac{\exp\bigl(-\beta_t \, \Delta L(\xi)\bigr)}{\sum_{\xi' \in \Xi_r(x)} \exp\bigl(-\beta_t \, \Delta L(\xi')\bigr)}. \tag{5}$$

When $\beta_t \to 0$ this is uniform random swapping; when $\beta_t \to \infty$ it concentrates on the best (most loss-decreasing) swap, recovering a soft version of the greedy stepwise algorithm. In practice we use a simple annealing schedule: β_t increases linearly from β_0 to β_1 over the allotted steps, gradually shifting from exploration to exploitation.

On the same standard example, the Zubarev walk also finds a solution, but much more slowly: 10,030 swaps for seed 0 on restart 0. That difference is consistent with its role here as a generic optimiser rather than a Sudoku-specific heuristic.

6 Experiments

We implemented both heuristics above in pure Python (see accompanying code). The prime was set to $p = 11$, but in the digit-restricted regime only the induced indicator behaviour matters. These experiments are small and purely illustrative; they are included to demonstrate that the encoded objective can be searched directly, not to establish a new benchmark for Sudoku solving.

6.1 Experimental setup

To obtain a quick, reproducible testbed, we generated random puzzles by:

1. sampling a random solved grid; then
2. *carving* it down to a specified clue count by removing entries uniformly at random.

Method	Clues	Puzzles	Solved	Median steps	Median time (s)
Stepwise	36	6	6	538	0.107
Stepwise	30	6	6	2785	0.734
Stepwise	26	6	6	7041	2.027
Zubarev walk	36	6	6	4534	0.800
Zubarev walk	30	6	6	8752	1.987
Zubarev walk	26	6	6	9912	2.929

Table 1: Heuristic performance on randomly carved puzzles (not uniqueness-checked), using $T = 60{,}000$ steps and $R = 15$ restarts. The Zubarev walk uses a linear schedule $\beta_0 = 0.5$ to $\beta_1 = 6.0$.

This procedure does *not* enforce uniqueness of the resulting puzzle (it is a fast stress test rather than a curated Sudoku benchmark). Each carved puzzle was then solved with up to $T = 60{,}000$ steps and $R = 15$ random restarts. All results reported below use a fixed RNG seed (123) for reproducibility.

6.2 Results

Table 1 summarises 36 runs (six puzzles at each clue count, for each heuristic). Both solvers succeeded on all instances in this small sample; runtimes are noisy because the puzzles are randomly carved, not difficulty-normalised, and the sample is too small to support stronger algorithmic claims.

Figure 3 shows the loss trajectory for the stepwise row-swap heuristic on the specific fast-carved 30-clue instance with puzzle seed 955101029 (solve seed 2638744256), using the row-wise random initialiser described above, $T = 60{,}000$, and $R = 15$. The plotted run is restart 0, and the loss is the number of conflicting pairs in columns and boxes (equivalently, the objective (2) up to an additive constant in the digit-restricted regime).

7 Representational parsimony

A standard SAT or exact-cover lift introduces Boolean variables

$$y_{r,c,d} \in \{0, 1\},$$

with the intended meaning that cell (r, c) contains digit d. This produces $9 \times 9 \times 9 = 729$ indicators before any clues are applied. By contrast, the p-adic construction keeps the problem in its native 81-cell digit space. This variable-level comparison is the main parsimony claim: one digit-valued parameter per semantic object rather than nine Boolean indicators per cell.

The minimal objective (1) is already a single scalar loss on that native state space. The expanded implementation (2) contains 729 digit-snapping residuals, $|\mathcal{C}|$ clue residuals, and 810 pairwise residuals. The point is not that this dominates every other formalism in raw constraint count: a structured constraint-programming model with 81 cell variables and 27 *alldifferent* constraints is already semantically close to the puzzle. Rather, the signed p-adic encoding packages the constraints into a single objective whose digit-restricted form is just a constant plus peer conflicts. In the row-permutation regime, the current search state is literally a Sudoku grid, and a proposed move can be audited by inspecting only the affected columns and boxes, as in Lemma 4. That is a claim about representational readability, not a theorem about runtime.

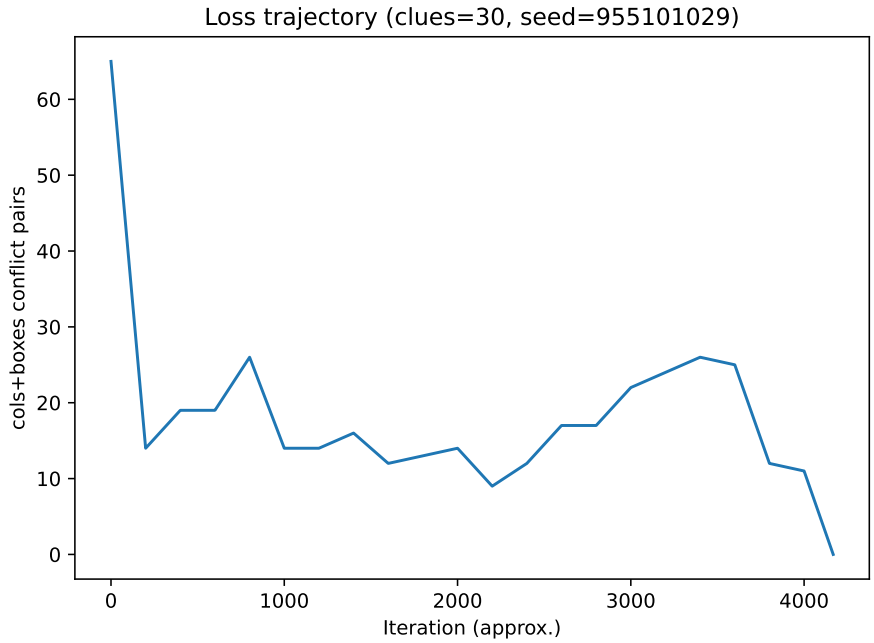

Figure 3: Loss/conflict trajectory for the stepwise row-swap heuristic on the fast-carved 30-clue instance with puzzle seed 955101029 (solve seed 2638744256), from the row-wise random initialisation on restart 0.

8 Discussion

The paper is not intended as a new Sudoku-solving method. Its contribution is representational: finite-domain all-different systems can be encoded as signed synthetic p-adic regression problems without leaving the native variable space. Sudoku is useful here because it makes both the construction and the local auditability of the resulting objective concrete.

Treating Sudoku completion as a regression instance still permits generic training dynamics such as the row-swap local search and the discrete Zubarev walk. Their role in this paper is illustrative rather than algorithmic. They show that one can search directly in the space of digit fillings while preserving the semantic transparency of the encoding.

An archived alternative maps digits to powers of two so that each row, column, or box contributes a single sum residual rather than a family of pairwise terms. That variant is structurally sparser, but it loses locality and was much less effective in preliminary experiments. The contrast is useful: mere constraint compression is not the point; the more successful construction is compact while remaining domain-native and locally auditable.

Several extensions are immediate:

- **Confusion-aware digit losses.** If clues came from OCR or handwriting rather than clean symbolic input, one could weaken the loss for commonly confused pairs (for example 1 versus 7) instead of treating every wrong digit as equally wrong.

- **Alternative constraint encodings.** One can replace the pairwise inequality reward with other synthetic regularisers, for example difference regularisation over the finite set $\{\pm 1, \ldots, \pm 8\}$, at the cost of a more complicated objective landscape.

9 Conclusion

This note shows that finite-domain all-different systems admit clean signed synthetic p-adic regression encodings, and then uses Sudoku as a vivid special case. The construction works because, for $p > q$, the p-adic norm turns every nonzero difference between values in $\{1, \ldots, q\}$ into the common value 1, so inequality becomes an indicator-like reward once the variables are domain-valued.

The main result is therefore theorem-driven rather than algorithmic. The encoding keeps the problem in the native 81-cell digit space rather than lifting it to 729 Boolean indicators, yields a single scalar objective whose digit-restricted form is just a constant plus peer conflicts, and supports local move audits in the original puzzle coordinates. Sudoku is only the case study. The broader point is that ultrametric regression can express finite-domain all-different systems in a compact, domain-native, and locally auditable way.

Acknowledgments

No specific acknowledgments are claimed in this version of the manuscript.

Funding

This research was supported by an Australian Government Research Training Program Scholarship.

Compliance with ethical standards

This article does not contain any studies with human participants or animals performed by the author.

Conflict of interest

The author declares that he has no conflict of interest.

Author contributions

This is a single-author paper. The author is responsible for the conceptual development, proofs, implementation, and writing.

References

[1] G. Baker, S. McCallum, and D. Pattinson. Linear regression in p-adic metric spaces. *p-Adic Numbers, Ultrametric Analysis and Applications* 17(4), 333–347 (2025). `https://doi.org/10.1134/S2070046625040016`

[2] G. Baker. NP-hardness of p-adic linear regression. Accepted for publication in *p-Adic Numbers, Ultrametric Analysis and Applications* (2026). arXiv:2602.13278

[3] Donald E. Knuth. *Dancing Links. Millennial Perspectives in Computer Science*, 187–214, 2000. Also available as arXiv:cs/0011047.

[4] Helmut Simonis. *Sudoku as a Constraint Problem.* In *Modelling and Reformulating Constraint Satisfaction Problems*, Fourth International Workshop, Sitges (Barcelona), Spain, October 1, 2005.

[5] Inês Lynce and Joël Ouaknine. *Sudoku as a SAT Problem.* In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics (ISAIM 2006)*, 2006.

[6] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals*, 86-A(5):1052–1060, 2003.

[7] A. P. Zubarev. p-Adic polynomial regression as alternative to neural network for approximating p-adic functions of many variables. *p-Adic Numbers, Ultrametric Analysis and Applications* 17(4), 413–420 (2025).